NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Knowledge for an Enlightening Future

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Backend Progress Report

ON

AI Programming Tutor & Bug Explainer

Course Code: CSE 4104

Course Title: Software Development III

SUBMITTED TO

Name: Md. Riaz Mahmud

Assistant Professor

Department of CSE

Northern University of Business & Technology Khulna.

SUBMITTED BY

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Date of Submission: 29/06/2026

Team Information

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Team Leader	M.M. Asfy Or Rayhan
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Contents

1. **Backend Technology Stack** ……..……..1

2. **Database Design Summary** ……...……...……..4

3. **Implemented APIs** ……..……..……...…….....…8

4. **Authentication Workflow** ……...……...……...……....…10

5. **Current Development Progress** ……..……...……..12

6. **Screenshots of API Testing** ……...……..14

7. **GitHub Repository Link** ……...……...……..21

Backend Progress Report

Project Title: AI Programming Tutor & Bug Explainer

1. Backend Technology Stack

The backend is a server-side JavaScript application built on Node.js using the ECMAScript module system, as declared by the "type": "module" field in server/package.json. The web framework is Express. Data persistence is provided by MongoDB through the Mongoose object-data-modelling library. The application entry point is server/src/server.js, and two npm scripts are defined: start (runs the entry point with Node.js) and dev (runs it under nodemon).

The dependencies declared in the backend package manifest are listed below.

Package	Version	Role in the Backend (as used in the source)
express	^4.19.2	HTTP server, routing, and middleware pipeline.
mongoose	^8.5.1	MongoDB connection and schema/model definitions.
jsonwebtoken	^9.0.2	Signing and verifying JSON Web Tokens.
bcryptjs	^2.4.3	Hashing and comparing user passwords.
cors	^2.8.5	Cross-Origin Resource Sharing configuration.
dotenv	^16.4.5	Loading environment variables (imported in server.js).
express-rate-limit	^7.4.0	Rate limiting on the AI, repo, and doc routes.
multer	^2.0.0	In-memory handling of uploaded files (doc route).

unpdf	^1.6.2	Extracting text from uploaded PDF files.
nodemon (dev)	^3.1.4	Automatic server restart during development.

Project Structure

The backend source is organised under server/src/ into clearly separated layers, as found in the archive:

Directory / File	Contents Found in the Project
config/db.js	MongoDB connection logic and a database-readiness helper.
models/	Mongoose schemas: User.js, ChatHistory.js, WeakTopic.js, Memory.js.
middleware/	auth.js (token verification, role guard) and errorHandler.js.
controllers/	authController, aiController, adminController, docController, repoController.
routes/	authRoutes, aiRoutes, adminRoutes, docRoutes, repoRoutes.
services/	aiService, githubService, memoryService, prompts.
server.js	Application entry point; assembles middleware and mounts routes.
.env.example	Template of required environment variables.

The server entry point applies the CORS middleware using the configured client URL, parses JSON request bodies up to a one-megabyte limit, and registers a rate limiter permitting at most thirty requests per minute on the /api/ai, /api/repo, and /api/doc route groups. A pluggable AI service abstracts the external model provider, selectable through an environment variable among

Groq, Gemini, OpenAI, and a built-in offline mock provider. A separate GitHub service reads public repositories through the GitHub REST interface. On the basis of the above, the backend project setup is present and configured, the framework and language are identified as Express on Node.js, and the structure is organised into discrete layers.

2. Database Design Summary

The database technology is MongoDB, accessed through Mongoose. The connection is established in config/db.js using the MONGO_URI environment variable. The connection is treated as optional in the code: when no URI is supplied, the application logs a warning and continues to run without persistence. Four collections are defined through Mongoose models. MongoDB assigns each document a default primary key field, _id (an ObjectId); no alternative primary key is declared in the schemas. Inter-collection relationships are expressed through reference fields (ref: 'User') rather than database-enforced foreign-key constraints. The implemented collections, their fields, and their validations are summarised below.

User Collection

Represents an account holder together with progress and subscription attributes.

Field	Type	Validation / Default (as declared)
name	String	Required; trimmed.
email	String	Required; unique; lower-cased; trimmed.
password	String	Required; minimum length 6; select: false (hidden by default).
role	String	Enum: student | admin; default student.
xp	Number	Default 0.
streak	Number	Default 0.
level	Number	Default 1.

lastActiveDate	Date	Optional.
subscription	String	Enum: free | pro; default free.
_id, createdAt, updatedAt	ObjectId / Date	Primary key and automatic timestamps.

A pre-save hook hashes the password with bcrypt (salt rounds 10) only when it has been modified. Instance methods provide password comparison and a sanitised JSON representation that removes the password field.

ChatHistory Collection

Stores individual interactions between a user and the AI features.

Field	Type	Validation / Default (as declared)
userId	ObjectId (ref User)	Required; indexed. Relationship: many records per User.
feature	String	Enum: tutor | bugExplainer | errorTranslator | codeImprover | problemGenerator; required.
message	String	Required.
response	String	Required.
language	String	Default 'en'.
topic	String	Optional.

_id, createdAt, updatedAt	ObjectId / Date	Primary key and automatic timestamps.

WeakTopic Collection

Records subjects with which a learner repeatedly struggles.

Field	Type	Validation / Default (as declared)
userId	ObjectId (ref User)	Required; indexed. Relationship: many records per User.
topic	String	Required.
mistakeCount	Number	Default 1.
confidenceLevel	Number	Default 0; constrained between 0 and 100.
(compound index)	userId + topic	Unique compound index — one record per (user, topic).

Memory Collection

Holds short factual notes that the tutor retains about a learner.

Field	Type	Validation / Default (as declared)
userId	ObjectId (ref User)	Required; unique; indexed. Relationship: one record per User.

notes	Array of sub-documents	Default empty array; each note has required text and a creation timestamp.
_id, createdAt, updatedAt	ObjectId / Date	Primary key and automatic timestamps.

In summary, four collections (User, ChatHistory, WeakTopic, Memory) are implemented. Primary keys are the default _id ObjectId fields. Reference relationships connect ChatHistory, WeakTopic, and Memory to User through the userId field, yielding one-to-many relationships for ChatHistory and WeakTopic and a one-to-one relationship for Memory (enforced by a unique constraint). Validations present in the schemas include required fields, enumerations, uniqueness constraints, length and numeric range limits, lower-casing, trimming, and a unique compound index. The schema definitions are consistent with the operations performed by the controllers.

3. Implemented APIs

The following endpoints are defined in the route files and mounted in server/src/server.js. They are grouped here as authentication, user/administration, and project-specific (AI, repository, document) endpoints. The access column reflects the middleware applied to each route: Public (no token), Protected (valid bearer token required), Admin (authenticated administrator required), and Optional (works for guests, with history and experience points recorded when authenticated).

Method	Endpoint	Access	Purpose (per controller)
GET	/api/health	Public	Report service status and the active AI provider.
POST	/api/auth/register	Public	Register a new account; returns a JWT.
POST	/api/auth/login	Public	Authenticate credentials; returns a JWT.
GET	/api/auth/me	Protected	Return the currently authenticated user.
GET	/api/ai/history	Protected	Return the user's 50 most recent activities.
POST	/api/ai/tutor	Optional	Explain a programming concept.
POST	/api/ai/bug-explainer	Optional	Explain bug(s) in submitted code and give a fix.
POST	/api/ai/error-translator	Optional	Translate a compiler/runtime error into plain language.
POST	/api/ai/code-improver	Optional	Suggest improvements to submitted code.

POST	/api/ai/problem	Optional	Generate a practice problem for a topic and difficulty.
POST	/api/repo/overview	Optional	Summarise a public GitHub repository.
POST	/api/repo/file	Optional	Explain a single file within a repository.
POST	/api/doc/explain	Optional	Explain an uploaded PDF or image document.
GET	/api/admin/stats	Admin	Return platform statistics and per-feature usage.
GET	/api/admin/users	Admin	Return the list of registered users.
GET	/api/admin/activity	Admin	Return recent activity across all users.
PATCH	/api/admin/users/:id/role	Admin	Promote or demote a specified user.

Categorisation Against the Evaluation Criteria

- **Authentication APIs:** POST /api/auth/register, POST /api/auth/login, and GET /api/auth/me.
- **User APIs:** user-related operations are limited to the authenticated-user endpoint (GET /api/auth/me) and the administrative user-management endpoints (GET /api/admin/users, PATCH /api/admin/users/:id/role). A separate general-purpose user-profile CRUD API is **Not Found in Uploaded Project**.
- **Project-specific APIs:** the AI endpoints (tutor, bug-explainer, error-translator, code-improver, problem, history), the repository endpoints (overview, file), the document endpoint (explain), the administrative reporting endpoints (stats, activity), and the health endpoint.

4. Authentication Workflow

Authentication is implemented and is based on stateless JSON Web Tokens. The relevant logic is contained in controllers/authController.js and middleware/auth.js.

User Registration

The registration handler first confirms that the database is connected, returning a 503 response otherwise. It validates that a name, email, and password are present (400 on absence), rejects an already-registered email (409), assigns the role admin if the email is listed in the configured ADMIN_EMAILS variable and student otherwise, persists the account, and returns a sanitised user object together with a signed token.

User Login

The login handler confirms database availability and the presence of credentials, retrieves the user while explicitly including the hidden password field, and compares the supplied password against the stored hash using a bcrypt-based method. On failure it returns a 401 response. If the email is configured as an administrator but the stored role is not yet admin, the role is upgraded and saved. On success it returns the sanitised user and a signed token.

Password Encryption

Passwords are encrypted using bcryptjs. A pre-save hook on the User model generates a salt (10 rounds) and hashes the password before storage, and only when the password has changed. The password field is excluded from queries by default (select: false), and a matchPassword method performs the hash comparison. Plain-text passwords are never persisted.

Token Issuance and Session Management

Tokens are issued by a helper that embeds the user identifier as the payload, signs it with the JWT_SECRET, and applies an expiry taken from JWT_EXPIRES_IN (defaulting to seven days). Authentication is therefore token-based and stateless; a server-side session store is Not Found in Uploaded Project.

Request Verification and Role Management

Three middleware functions govern access. The protect guard extracts and verifies the bearer token, loads the corresponding user, and returns 401 on any failure. The optionalProtect guard performs the same verification but never blocks the request, attaching the user when a valid token is present and otherwise treating the requester as a guest. The adminOnly guard, applied after protect, permits a route only when the user's role is admin and returns 403 otherwise. Role management is implemented through the student/admin enumeration, the environment-driven administrator list, and the administrative endpoint that updates a user's role.

5. Current Development Progress

The current backend implementation establishes the core foundation of the AI Programming Tutor & Bug Explainer system. Based on the uploaded source code, several essential backend components have been successfully implemented, including server configuration, database integration, authentication, AI-powered services, repository management, document processing, memory management, administrative features, and provider abstraction. The following sections summarize the backend functionality currently implemented in the project.

Implemented Backend Functionality

The backend is built using Express.js and Node.js with a layered architecture that includes CORS support, JSON body parsing, and rate limiting for AI, repository, and document-related routes. MongoDB is integrated through Mongoose, with four collections—User, ChatHistory, WeakTopic, and Memory—and a fault-tolerant database connection mechanism. The authentication system provides user registration and login with bcrypt password hashing, JWT-based authentication, and role-based access control for student and administrator users. The application includes several AI-powered services, such as a programming concept tutor, bug explainer, compiler and runtime error translator, code improvement assistant, and practice problem generator. It also provides a GitHub repository service capable of retrieving repository metadata, README files, repository structure, and individual file contents using the GitHub REST API. In addition, the backend supports document explanation by accepting uploaded PDF files for text extraction and image files for processing through a vision-capable AI model, with uploads handled entirely in memory. A memory management service is implemented to store user notes and weak programming topics, allowing contextual information to be reused in future AI interactions. Administrative functionality is also available, including platform statistics, feature usage reports, user management, recent activity monitoring, and user role updates. The AI service is designed with a provider abstraction layer that supports Groq, Gemini, OpenAI, and an offline mock provider.

Database Connectivity

The backend APIs communicate with the MongoDB database through Mongoose models. User account information, AI interaction history, and memory notes are stored in the database whenever the database connection is available. Stored information can be retrieved through history endpoints, administrative statistics, user management endpoints, activity monitoring, and memory context retrieval. Database-dependent operations are protected by an isDBReady verification mechanism, while history-saving operations are executed within try/catch blocks to prevent unexpected failures. When the MONGO_URI configuration is unavailable, the server continues running, but database-dependent features such as account management, administrative operations, and history storage return a 503 Service Unavailable response.

Error Handling

The backend implements comprehensive error handling to ensure system stability during invalid or unexpected requests. Validation errors return HTTP 400 responses whenever required request fields are missing. Authentication failures produce HTTP 401 responses for missing or invalid tokens, non-existent users, or incorrect login credentials, while administrator-only routes return HTTP 403 responses for unauthorized users. Database connection failures are handled gracefully by detecting unavailable connections and returning HTTP 503 responses for database-dependent operations. A centralized error-handling middleware also manages API and file upload exceptions by returning HTTP 404 for undefined routes, HTTP 413 for oversized uploads, HTTP 415 for unsupported file formats, HTTP 422 for PDF processing failures, and HTTP 500 for unexpected server errors. When external AI provider requests fail, the application automatically switches to the offline mock provider to maintain service availability.

API Testing

No Postman collections, Thunder Client collections, .http or .rest request files, automated API test files, or API testing screenshots were found in the uploaded project archive. Therefore, API testing artifacts are Not Found in Uploaded Project.

6. Screenshots of API Testing

This document compiles the API testing evidence for the backend of the "AI Programming Tutor & Bug Explainer" project. The endpoints were exercised with the Thunder Client REST client against the locally running backend server at http://localhost:5000, while the database was hosted on MongoDB Atlas. The screenshots are organised into five groups: backend folder structure, database collections, the authentication process, general API testing results including error handling, and the GitHub repository. Each figure is captioned with the request, the observed HTTP status, and the response, exactly as shown in the screenshot.

6.1 Backend Folder Structure

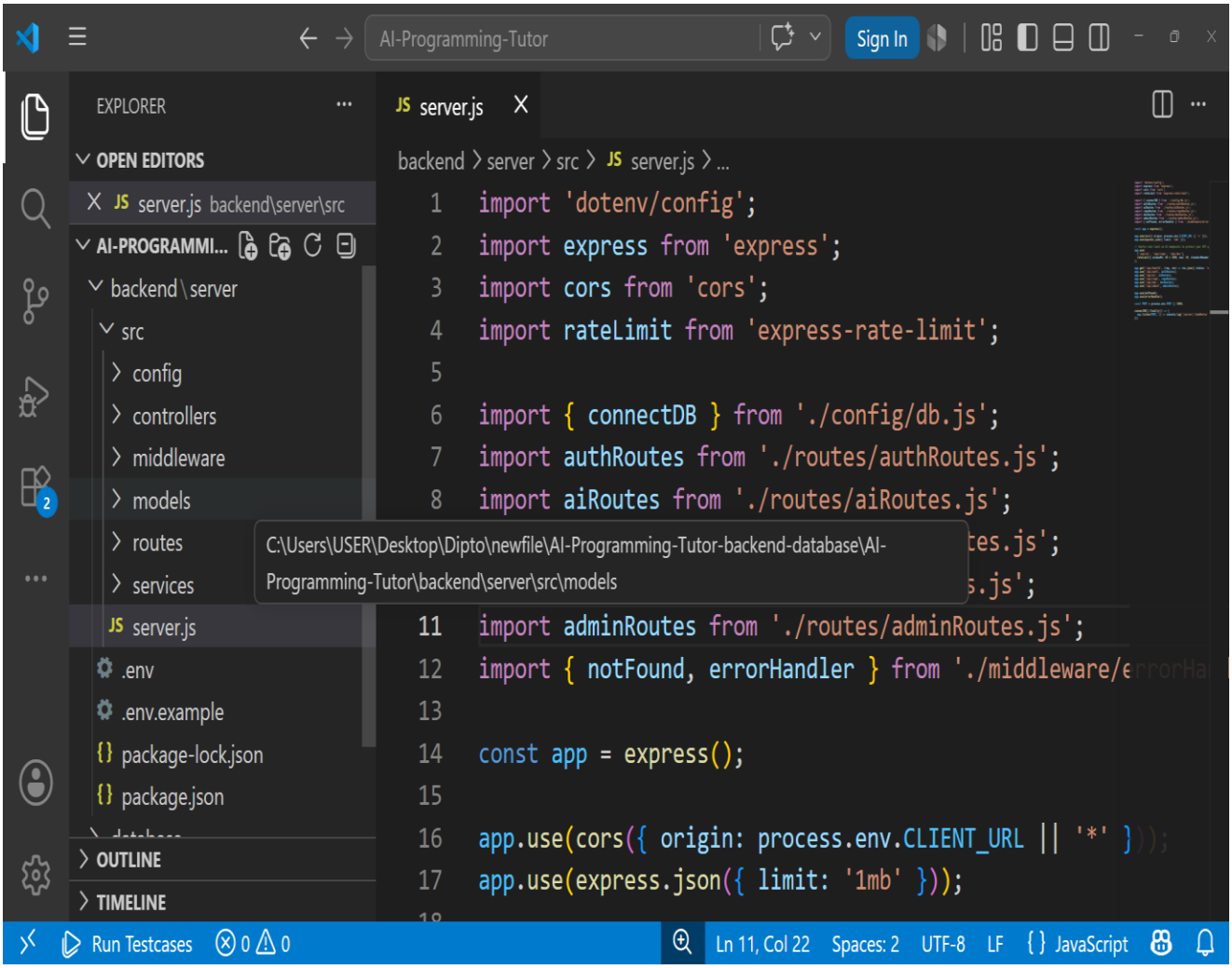


Figure 1: Backend Folder Structure

6.2 Database Tables/Collections

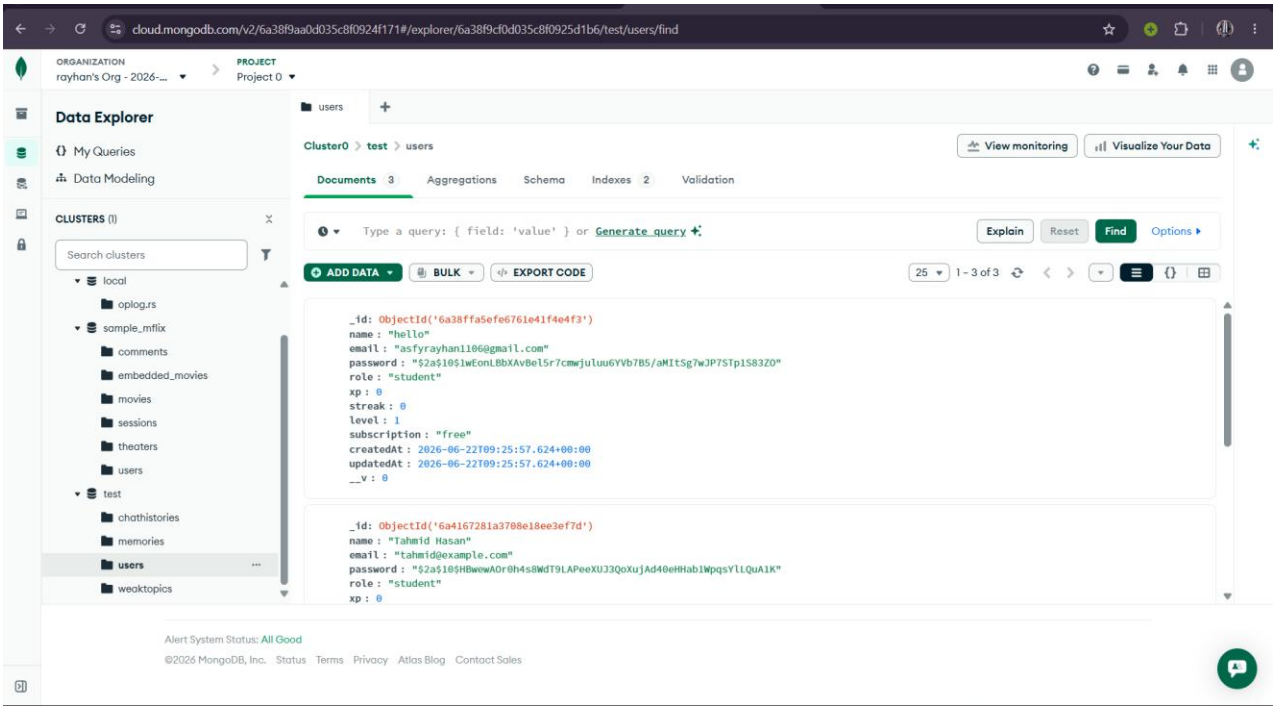


Figure 2: MongoDB Collections

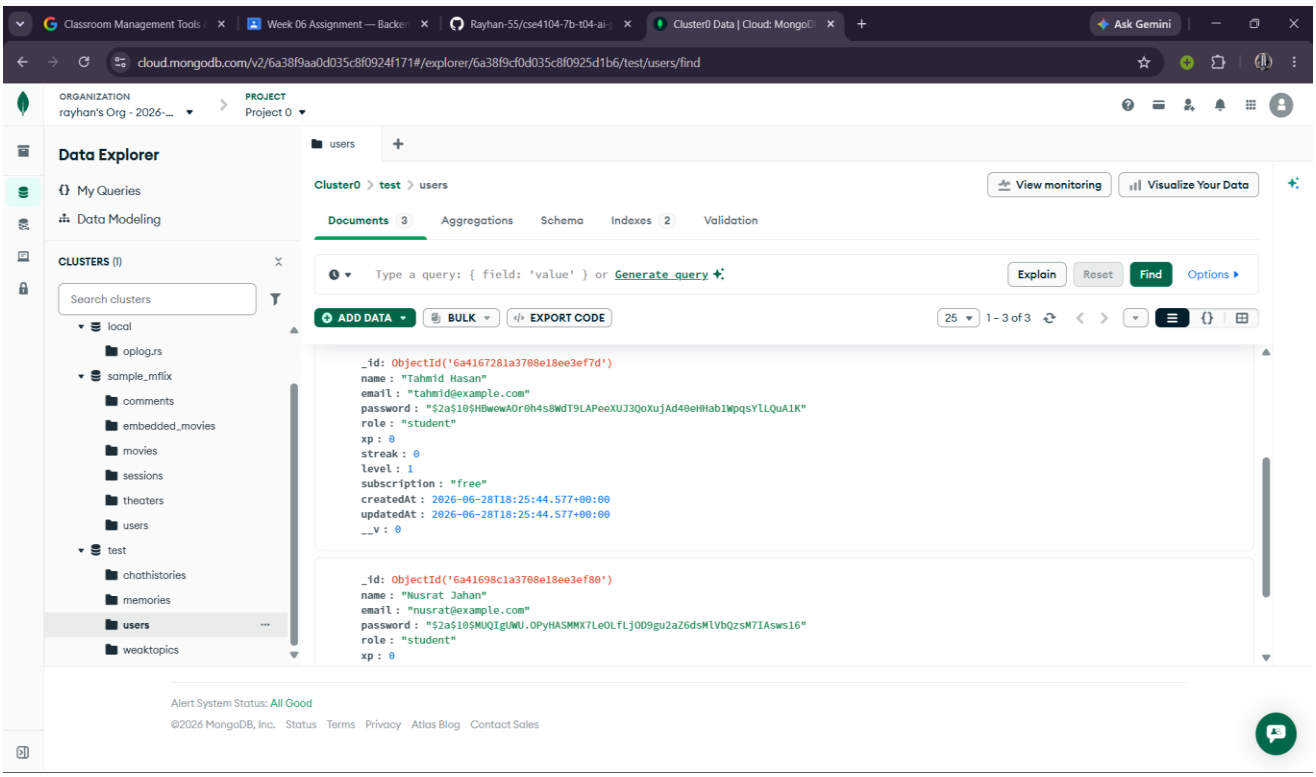


Figure 3: User Documents

6.3 Authentication Process

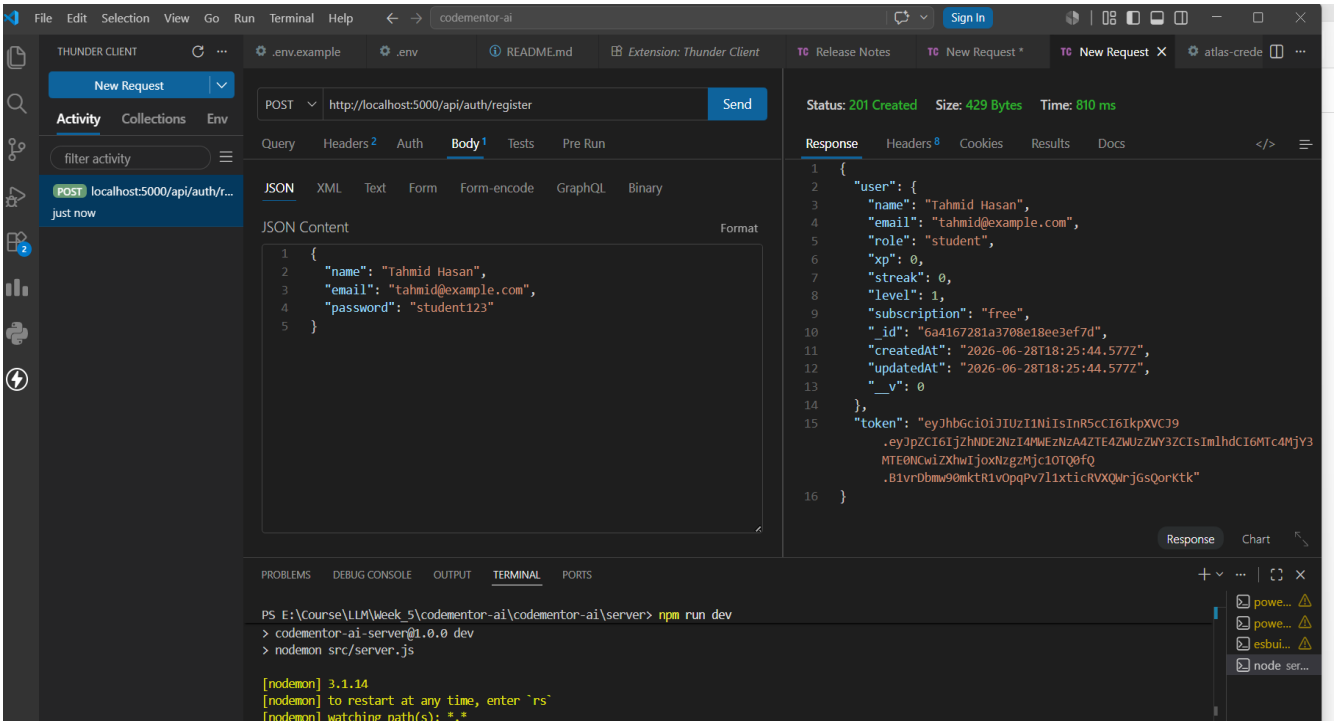


Figure 4: User Registration (POST /api/auth/register)

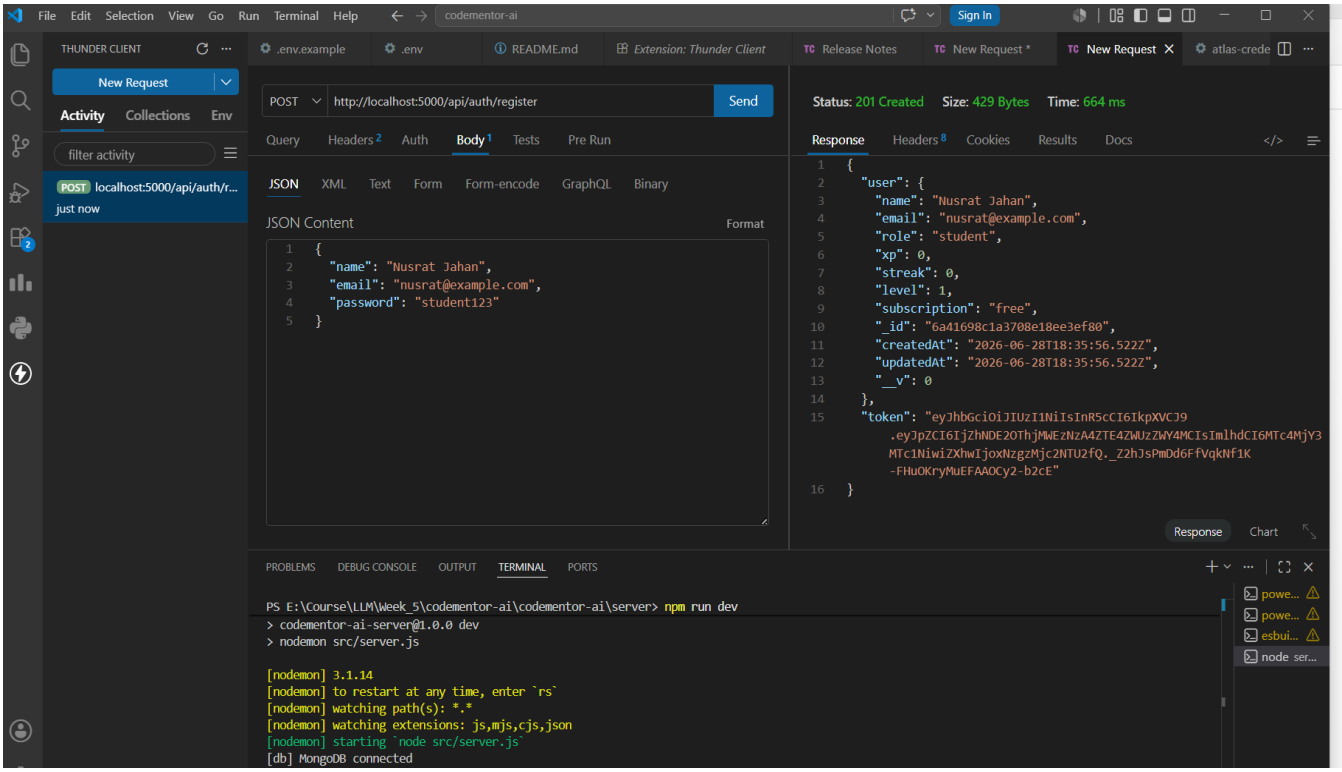


Figure 5: Registration Confirmed in Database

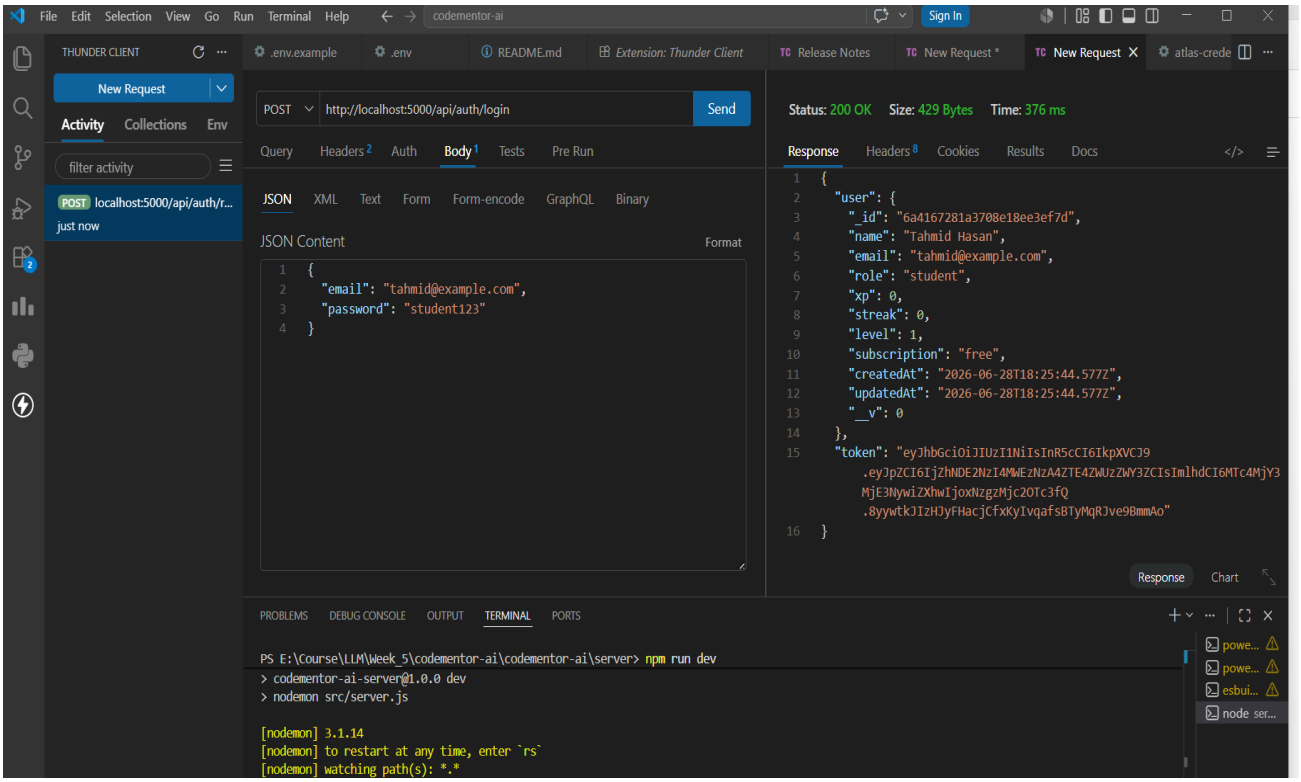


Figure 6: User Login (POST /api/auth/login)

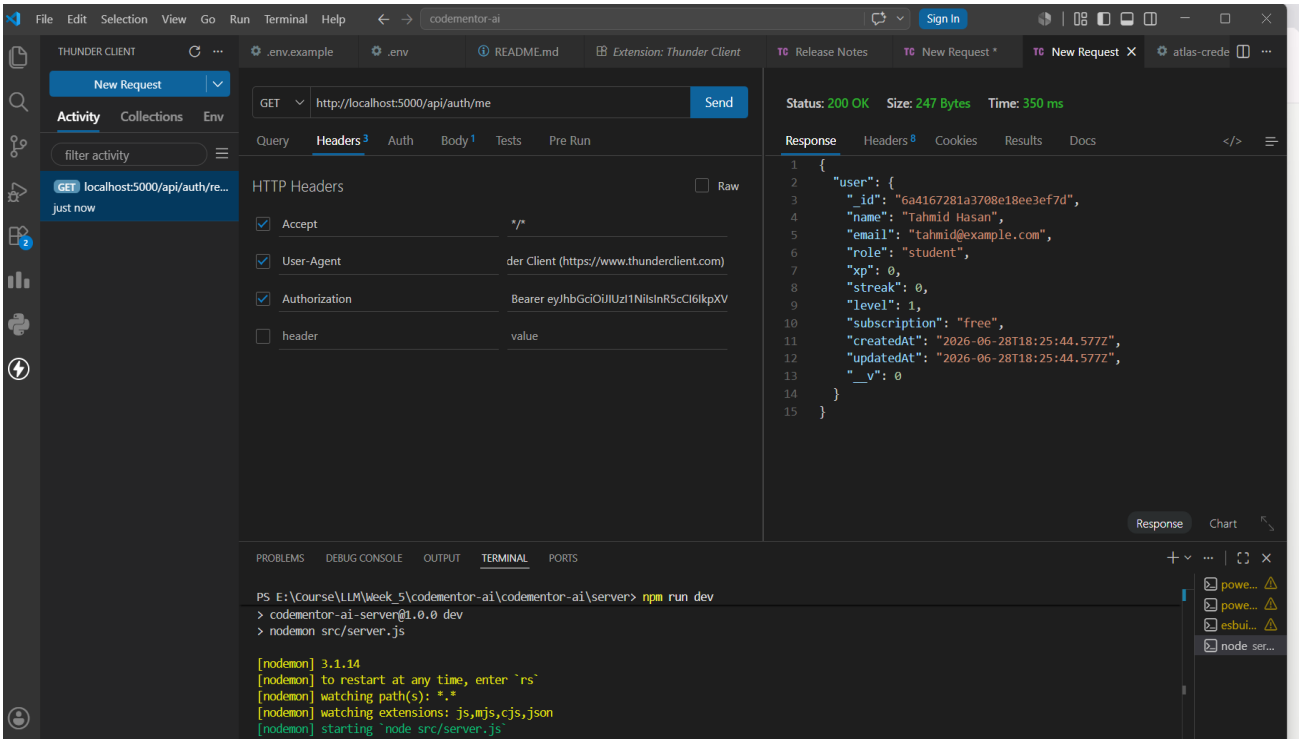


Figure 7: Protected Route (GET /api/auth/me)

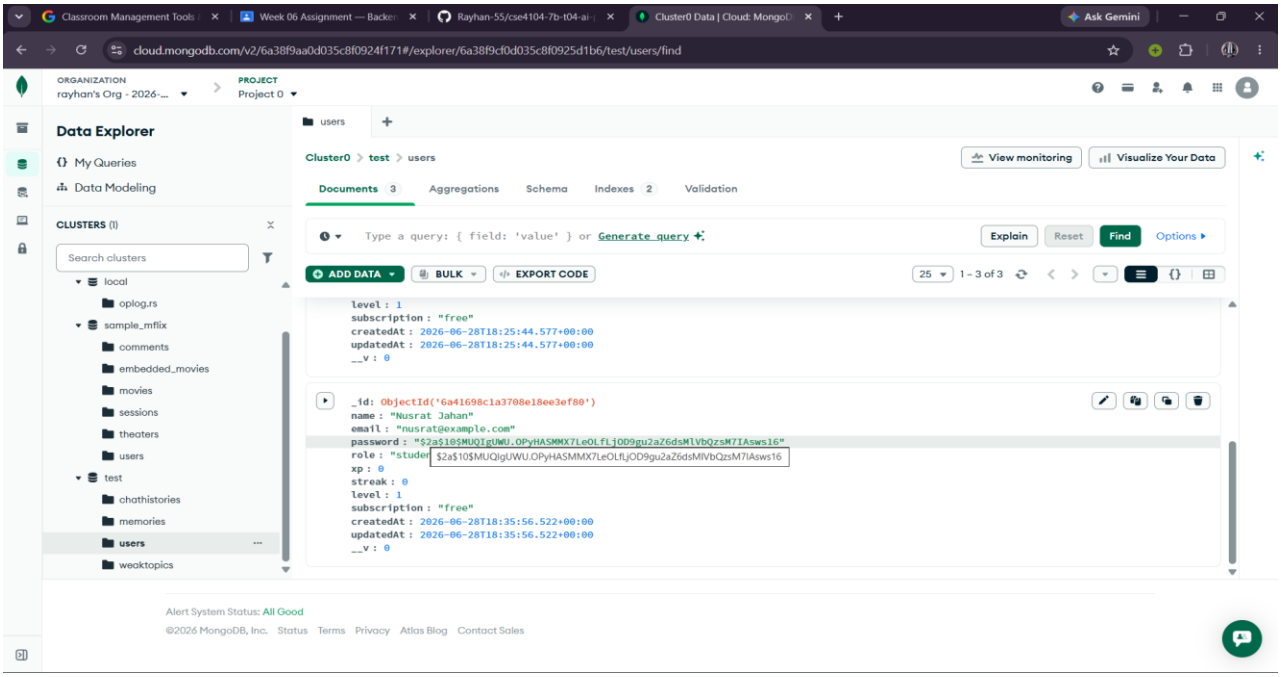


Figure 8: Password Encryption

6.4 API Testing Results

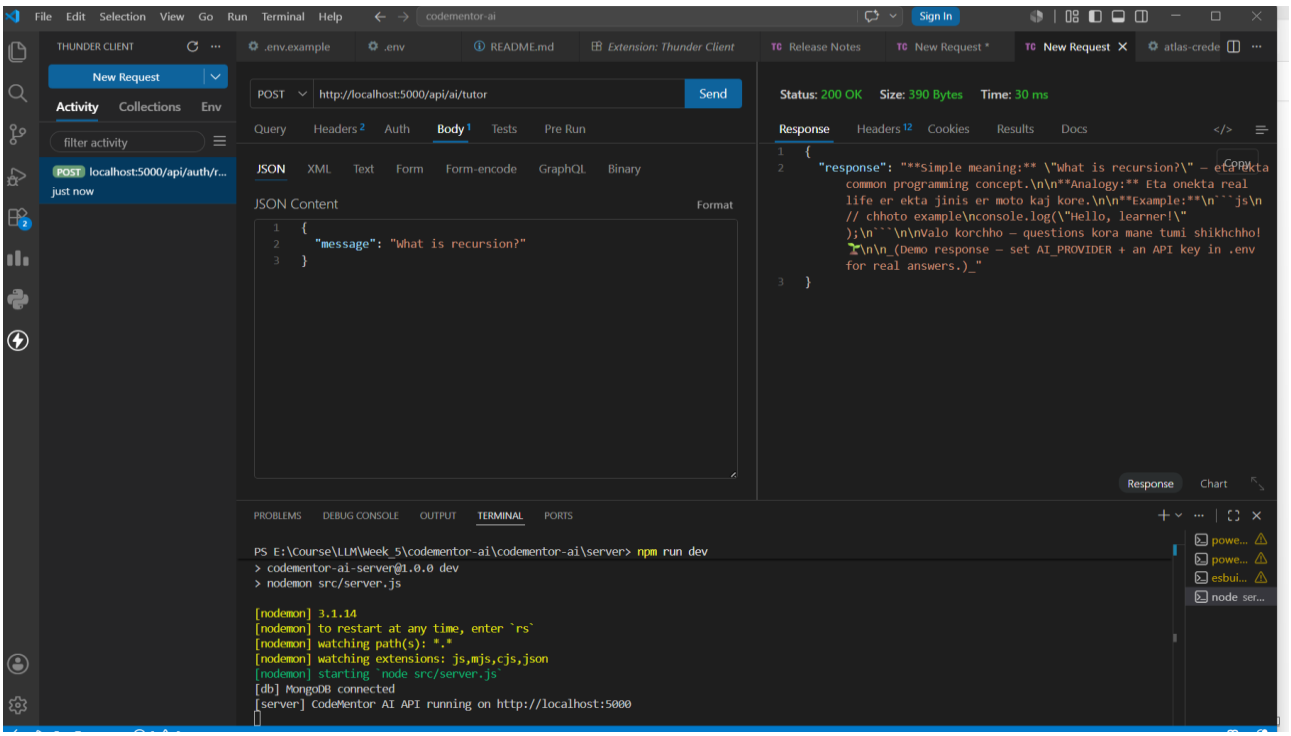


Figure 9: AI Tutor Endpoint (POST /api/ai/tutor)

Error Handling

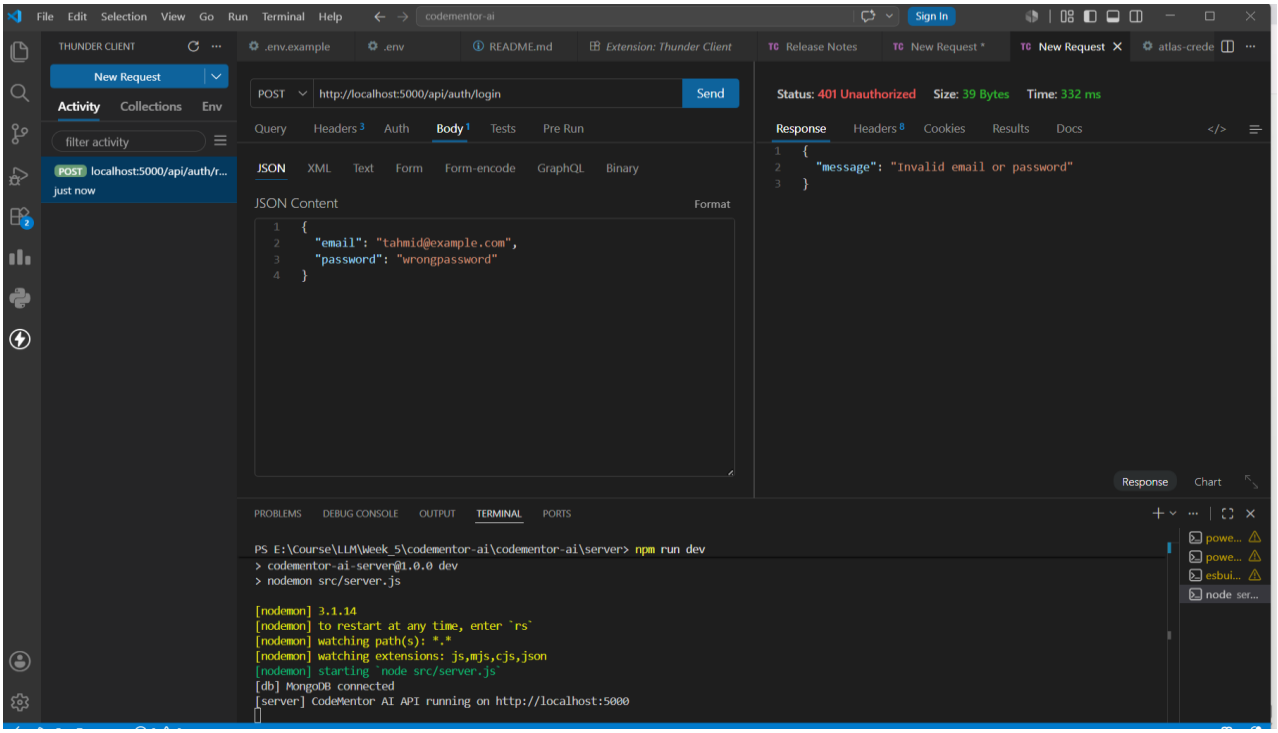


Figure 10: Invalid Credentials (401)

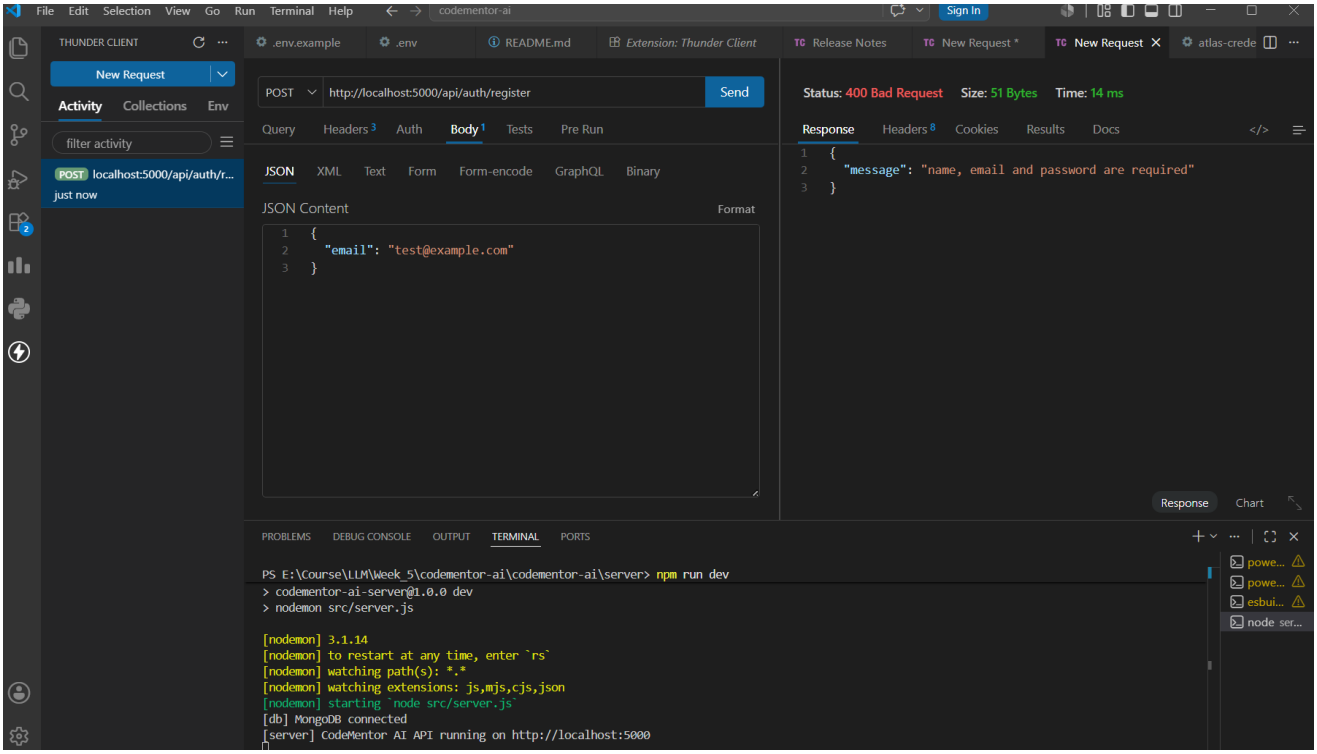


Figure 11: Missing Required Fields (400)

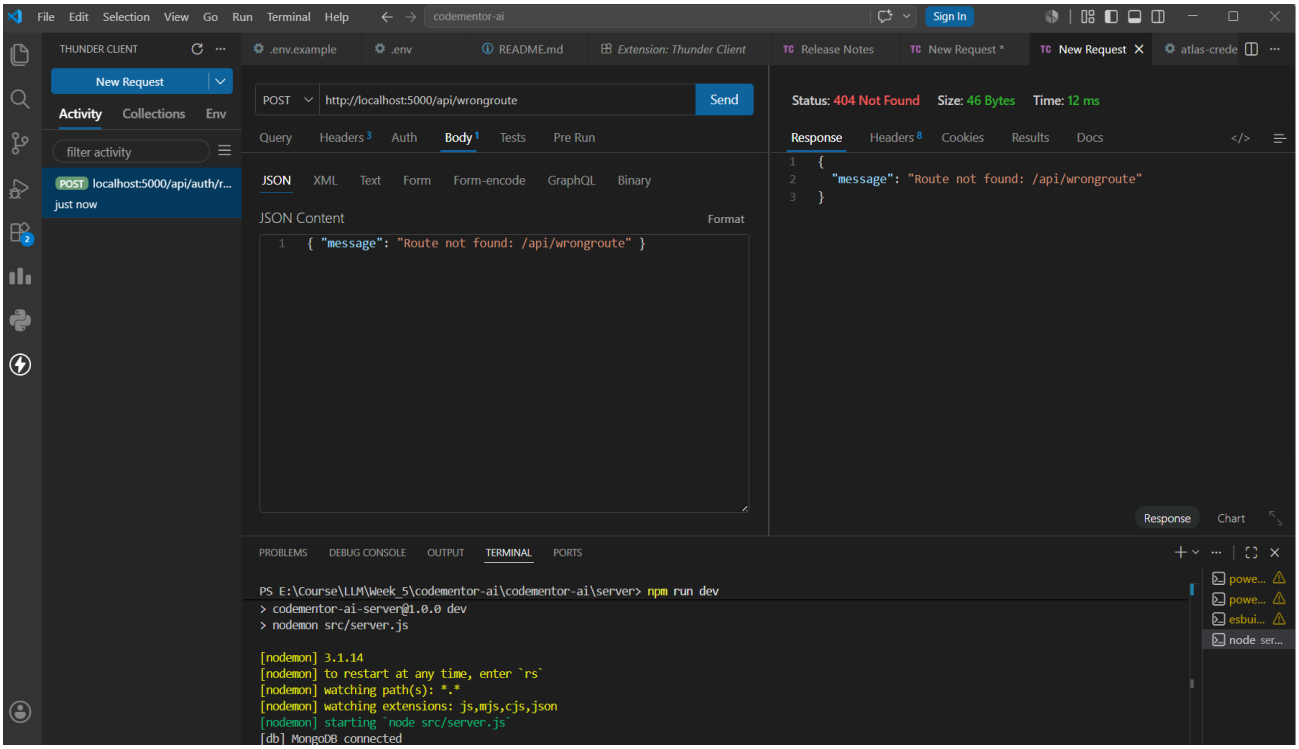


Figure 12: Unknown Route (404)

6.5 GitHub Repository

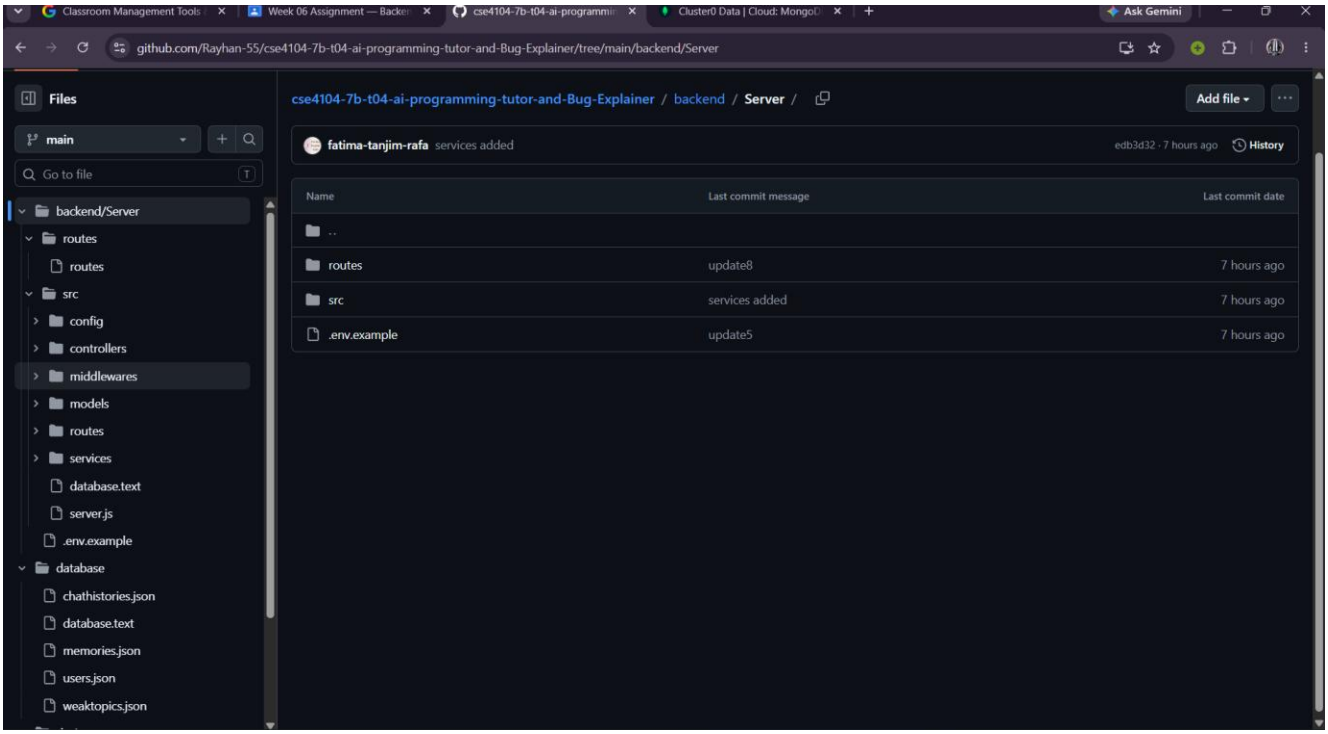


Figure 13: Project Repository on GitHub

7. GitHub Repository Link

https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer